CE152: Object-Oriented Programming

Inheritance

Dr. Sebastian Halder

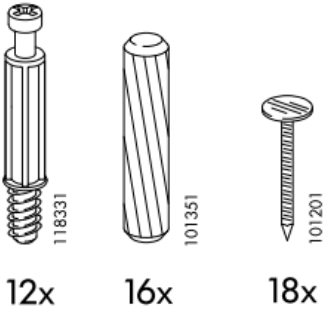
CSEE

2023

The idea behind object-oriented programming

Building with pre-defined objects

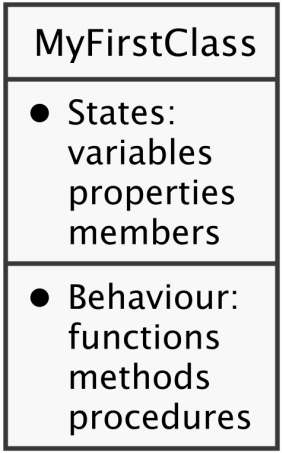
Creating Objects for programming



- We use basic building blocks and combine them into more complex structures
- The main idea is to break a complicated problem into small, manageable, reusable chunks
- These chunks can be reused so that we don't have to "reinvent the wheel"
- For example, all items on the left are *connectors* and must therefore interface with two other Objects

Inheritance

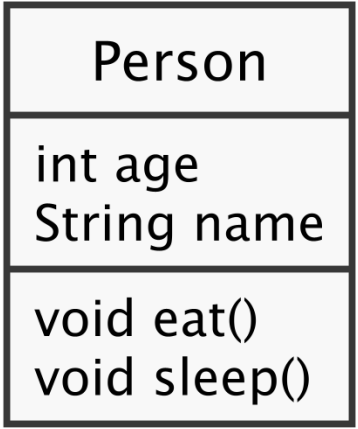
The first Unified Modelling Language (UML) diagram



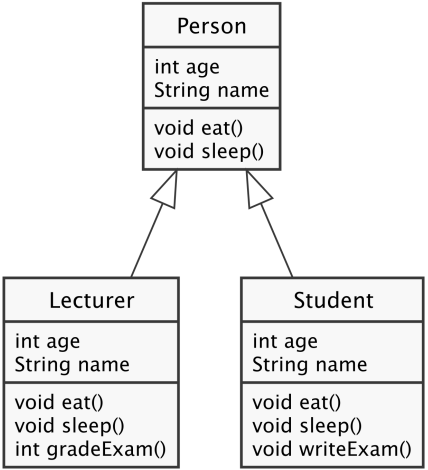
UML class diagrams

- On the previous slide you saw a *Unified Modelling Language (UML) diagram*
- UML is a standardised set of diagrams to describe software independently of a programming language
- These can be used to describe the properties of a class
  - The name (in this case MyFirstClass)
  - The states (class variables)
  - The behaviour (method; essentially a function that belongs to a class)
- We will be using UML mostly for so-called class diagrams
  - https://en.wikipedia.org/wiki/Class_diagram

Person class definition



Principles of OOP: Inheritance



What is inheritance good for?

- The arrows in the UML diagram indicate *inheritance*
  - Student inherits the methods from Person and adds writeExam
  - Lecturer inherits the methods from Person and adds gradeExam
- This allows us to reuse the code from Person increasing our efficiency in writing and maintaining the code
  - For example if I want to change the sleep() method I only need to do so once
  - If I want to add a variable birthday to person I only need to do so once

Defining a class

- This creates a class called Person
- It does nothing

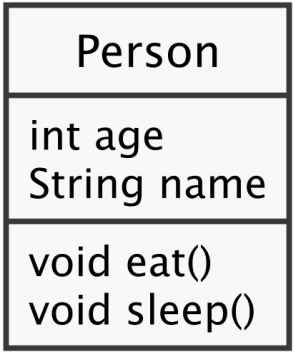
```
class Person{

}
```

- We could instantiate it using the standard constructor
- The constructor is what follows the new keyword

```
Person myPerson = new Person();
```

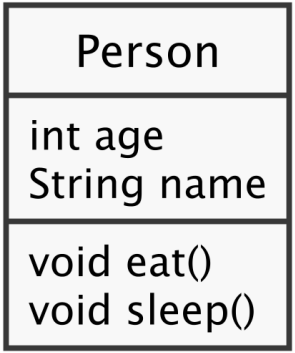
The constructor



- The constructor is the method we use to create an Object based on the class definition
- The default has no parameters and you do not have to declare it like I did below because Java does this for you

```
class Person{
  Person(){
  }
}
```

Adding member variables

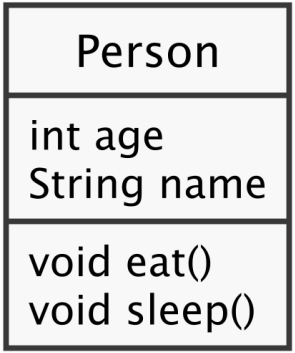


- We add the member variables above the constructor
- We could add them anywhere but it is good practice to put them in one place
- If we do not initialise them they will contain null

```
class Person {
    int age;
    String name;

    Person(){
    }
}
```

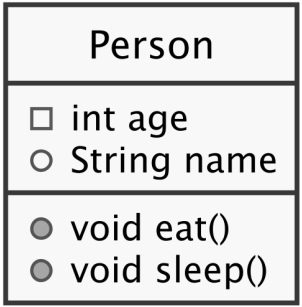
Initialising a Person



- The idea is to make a constructor that initialises our Person to the state you want (the default constructor is now gone!)
- Our Person will be provided with a name we choose and the age will be set to 0

```
class Person {
    int age;
    String name;
    //the this keyword ensures that
    //we assign to the member variable
    Person(String name){
      this.name = name;
      this.age = 0;
    }
}
```

Adding a method

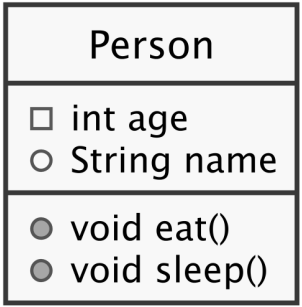


- When declaring a method we need:
  - The access modifier (can be blank which will be the default (visible in package))
  - The return type (can be void or any primitive or Object)
  - The name (eat, sleep)
  - The parameters (in the (), can be blank or any number of primitives or Objects)

```
void eat(){
}

void sleep(){
}
```

Adding functionality to the method

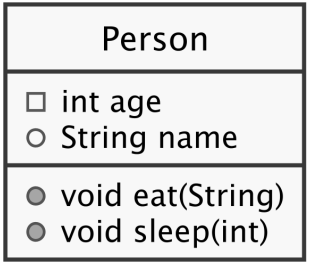


- The methods on the previous slide would not do anything when called
- Let's add some output

```
void eat(){
  // This will print to
  // the command line
  System.out.println("omnomnom");
}

void sleep(){
  System.out.println("zzZzzZzzZzz");
}
```

Adding parameters to the method



- Parameters are added with types and variable names inside the parentheses
- These can now be accessed inside the methods
- Strings can be concatenated with "+"

```
void eat(String food){
    System.out.println(
        "omnomnom " + food);
}

void sleep(int hours){
    System.out.println(
        hours + " hours of zzZzz...");
}
```

Returning values

Person
□ int age ○ String name
● void eat(String) ● void sleep(int) ● int getAge()

- getAge() returns the int age
- This means when we call the method we are provided the age and can process it in some way
- We now increase age by one everytime the Person sleeps (age++ is equal to age = age + 1)

```
int getAge(){
    return age;
}

void sleep(int hours){
    System.out.println(
        hours + " hours of zzZzz...");
    // the ++ operator increments by 1
    age++;
}
```

Some additional terms

- We call what is between the $\{\}$ of the method the *body* of the method
- Variables we create inside the method *body* exist only there and are deleted when we leave the method
- They variables inside the *body* are *local variables*
- We may also refer to the method body as a *block*

The complete Person (1)

```
// We create a class called Person
class Person {
    // With the member variables age and name
    int age;
    String name;
    // In the constructor the name is set by a parameter
    // and the age is set to zero
    Person(String name){
        this.name = name;
        this.age = 0;
    }
    // Our person will eat what we pass as a String
    void eat(String food){
        System.out.println(
                "omnomnom " + food);
    }
```

The complete Person (2)

```
    // continued from previous slide
    void sleep(int hours){
        System.out.println(
                hours + " hours of zzZzzZzzzZzz");
        // increases age by one when Person sleeps
        this.age++;
    }

    // tells us the persons age
    int getAge(){
        return this.age;
    }
// This } closes the first curly bracket of the class
}
```

Deriving Lecturer from Person (1)

```
class Lecturer {
/*
This code will create a new class
called Lecturer with a default
constructor. It will not inherit
anything from Person.
*/

}
```

Error due to not declaring inheritance

```
class mainExercise1 {
    static void main(String args[])
    {
      // We could now do this:
      Lecturer myLecturer = new Lecturer();

      // The following would result in errors:
      myLecturer.sleep(6);
    }
}
```

Deriving Lecturer from Person (2)

```
class Lecturer extends Person {
    // Note the extends keyword!
    // Why are we declaring a constructor?
    Lecturer(String name){
        // Note the super keyword!
        super(name);
    }
}
```

Inheritance in Java

- We use the "extends" keyword to show *inheritance*
- We call the inheriting class *subclass* or *child* (in this case Lecturer is the subclass/child)
- We call the class being inherited from *superclass* or *parent* (in this case Person is the superclass/parent)
- With "super" we can call methods of the *superclass*

Constructors and inheritance (1)

- You can invoke a parent class constructor by using the keyword `super` followed by appropriate arguments
  - This must be the first statement in the subclass constructor
- If a subclass constructor does not call `super` explicitly, then the parent class must have a zero-argument constructor
  - This is invoked implicitly at the start of the subclass constructor
  - In our case there is no zero-argument constructor thus we must use `super`

```
//zero argument constructor
//no need for super in sub-classes
MyClass(){...}
//constructor with arguments
//if we have only this constructor we need super
MyClass(int x){...}
```

Constructors and inheritance (2)

- It follows that whenever you create an object of a subclass, a parent class constructor will also be called
- This in turn may trigger a chain of further super-class constructors until the *parent class of all Java classes* is reached which is called `Object`

Using inherited methods

```
class mainExercise1 {
    static void main(String args[])
    {
      Lecturer myLecturer = new Lecturer("Sebastian");
      // Now this works! What will be the output?
      myLecturer.sleep(6);
    }
}
```

So why did we do all this?

- Currently Lecturer does exactly what Person does
- We can now add new methods or *override* existing ones
- We can also add new member variables

The original sleep method

```
...
void sleep(int hours){
    System.out.println(
            hours + " hours of zzZzzZzzzZzz");
    // increases age by one when Person sleeps
    this.age++;
}
...
```

Overriding a method

```
...
@Override
void sleep(int hours){
    System.out.println("No rest for the lecturer...");
    // ages twice the speed of Person
    this.age = this.age + 2;
}
...
```

- We use `@Override` to tell the compiler that we want to replace the method in the parent class
- It is **not** necessary and if we make no mistakes we can achieve the same effect without `@Override`
- If we make a mistake though the compiler will inform us if there is no parent class method to override

The two sleep() methods

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myPerson.sleep(12);
System.out.println(myPerson.name + " is "
+ myPerson.getAge()
+ " undefined units of time old...");
myLecturer.sleep(6);
System.out.println(myLecturer.name + " is "
+ myLecturer.getAge()
+ " undefined units of time old...");
```

- Output:

```
12 hours of zzZzzZzzzZzz
Winston is 1 undefined units of time old...
No rest for the lecturer...
```

Adding a new method

```
class Lecturer extends Person{
    Lecturer(String name){
        super(name);
    }

    void sleep(int hours){
        System.out.println("No rest for the lecturer...");
        this.age = this.age + 2;
    }

    double gradeExam(String exam){
        // Math.random() is an rng from 0 to 1
        // What keywords before random() in class def?
        return Math.random()*100;
    }
}
```

Grading exams

```
System.out.println("Your grade is " + Math.round(
  myLecturer.gradeExam("Inheritance...")));
System.out.println("Your grade is " +
  myLecturer.gradeExam("Abstraction..."));
System.out.println("Your grade is " +
  Math.round(myLecturer.gradeExam("Polymorphism...")));
System.out.println("Your grade is " +
  Math.round(myLecturer.gradeExam("...")));
```

- Why does one grade have more decimals?

```
Your grade is 47
Your grade is 6.7928288997513135
Your grade is 73
Your grade is 78
```

Rules for casting objects

- You can always *upcast* an Object, you can only *downcast* an Object if this is the original type
- We can do this

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myPerson = myLecturer;
```

- But not this!

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myLecturer = myPerson;
```

Up- and downcasting

- The reason is that the `Person` may not have all the variables of the `Lecturer` while the `Lecturer` will always have all the variables of `Person`
- Upcasting (up the inheritance tree) is always ***safe***
- Sometimes we may need to downcast (down the inheritance tree)
- ***Remember:*** if we cast an object to another type it will still be the original object!
- We can determine the class using `getClass()`

```
String x = "CE152";
Object b = (Object)x;
System.out.println(b.getClass()); // This would output
                                  // class java.lang.String
```

instanceof

- Then we can `instanceof` to determine the type of an Object during runtime
- For this purpose we can use the *instanceof* operator
- Use of this operator when it is not absolutely necessary is *considered bad practice*
- We should be certain that we cannot accomplish the *same effect by overriding* the method

```
String x = "CE152";
if (x instanceof String) { // This would be true
  ...
}
```

When to use instanceof

- Again, if we find ourselves using instanceof frequently, we may want to redesign our code
- Nevertheless, instanceof can be useful for example when we receive input from a class we do not have access to or by user input
- Then we may want to determine if the input we receive is e.g. a String, Integer or another type of Object

Live coding

- Create a class `Person`
- Create a class `Student` that extends/inherits from `Person`
- Demonstrate how functionality is replicated

Summary

Defining a class in Java

```
class MyClass {}
```

Defining inheritance in Java

```
class MyClass extends MyParentClass{}
```

Adding a method to a class

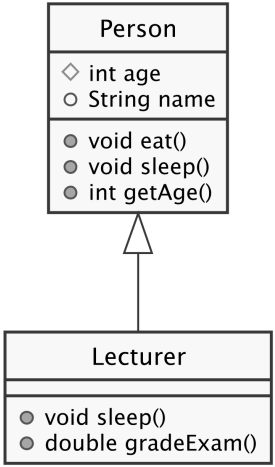
```
class MyClass extends MyParentClass{
  // constructor (does not return anything)
  MyClass(){}
  // method  (returns void or a variable)
  void myMethod(){}
}
```

The program has to start somewhere

```
//We could also put the main method into MyClass
class MyMain{
  static void main(String[] args){
    MyClass myVariable = new MyClass();
    myVariable.myMethod();
  }
}
```

- ***It is critical that you understand all of the code on this slide and the previous three slides***

Our UML diagram



When to use inhertiance?

- This is a difficult question
- Primarily inheritance should be used in an additive way
- The class `Lecturer` should be used like the class `Person` plus something extra
- If there are some aspects of the superclass you do not want in your subclass inheritance may not be the best approach
- For example, you could extend your class from `String` instead of adding a variable of type `String`

Summary

- We learned about one of the four principles of OOP: ***inheritance***
- We use the *class* keyword to declare our class definition
- We use the *extends* keyword to declare inheritance
- We run our code from *static method* called main() from a separate file

Thank you and see you in the next video!